Technische Universität Dresden

Faculty of Computer Science, Chair of Systems Engineering

A Comparative Analysis of WebAssembly and Docker Containers for Microservice Architectures in Kubernetes

Abdulaziz Musaev

Born on 22nd November, 1999 in Denau, Uzbekistan

Master's Thesis in Distributed Systems Engineering

4 March 2026

Advisor

Dr. Michael Steininger

Referees

Prof. Dr. Christof Fetzer

Dr.-Ing. André Martin

Erklärung

Ich versichere, dass ich alle bewertungsrelevanten Beiträge dieser Arbeit ohne Hilfe anderer Personen und Werkzeuge erbracht habe oder diese Beiträge klar und unmissverständlich im Text gekennzeichnet habe (etwa durch Zitate). Diese Arbeit wurde in gleicher oder ähnlicher Form keiner anderen Prüfungsbehörde vorgelegt. Sollte ich mehrere Kopien dieser Arbeit zur Bewertung abgeben, so ist deren Inhalt identisch.

Declaration

I declare that I have achieved all grading-relevant contributions to this work without the help of other persons or tools, or that I have clearly and unmistakably identified such contributions in the text (e.g., by citations). This work has not been submitted in the same or a similar form to any other examination body. Should I submit multiple copies of this work for grading, their content is identical.

(Abdulaziz Musaev)
Dresden, 4 March 2026

Abstract

WebAssembly (Wasm) is a binary instruction format originally designed for web browsers that provides a portable, high-performance execution environment. It enables web browsers to execute code written in multiple languages seamlessly, delivering high performance across different platforms. While Wasm's use on the client side is well known, with the introduction of the WebAssembly System Interface (WASI), its deployment on the server side has recently gained traction, especially in containerized environments managed in Kubernetes clusters. However, integrating Wasm into container runtimes presents challenges related to concurrency, efficient use of multi-core CPU architectures, developer experience, and ecosystem maturity.

While Docker remains the de facto standard for containerization in Kubernetes environments, its relatively high resource consumption and startup times can be limiting, especially in scenarios that require rapid scaling or high efficiency. In contrast, WebAssembly promises a more lightweight execution model, with near-instant startup and reduced memory usage.

This thesis explores the memory efficiency/footprint, cold and warm start-up times, security and performance trade-offs introduced by running Rust code in a Wasm runtime, the ecosystem maturity, and the development experience of Wasm versus conventional Linux containers deployed with Docker. It also analyzes maintainability and operational complexity for real-world cloud deployments and provides recommendations and best practices for adopting WebAssembly in cloud-native environments.

The findings of the thesis indicate that Wasm containers feature significantly lower image sizes than Docker containers and can improve cold-start times considerably. However, due to Wasm's lack of multithreading support, the application's performance degrades significantly, leading to increased latencies and reduced throughput compared to conventional containers. Furthermore, the Wasm execution model requires including only those libraries that can be compiled to WASM binary, since not all libraries, third-party packages are written in Rust language (those that heavily rely on C, C++ standard library) can not be fully compiled to WASM binary, which limits the number of libraries at our disposal, while developing with WebAssembly. Plus, with the Rust programming language notoriously known for its steep learning curve, it adds an extra layer of difficulty to developing and maintaining WASM-based projects. Despite the growing adoption of Rust, the small pool of proficient Rust and WebAssembly experts remains limited, with most remaining primarily in research and academia. Nevertheless, the current technology limitations of Wasm still present more challenges than what it actually brings to the table compared to conventional Docker containers when it comes to developing next high-scale projects in the current rapidly evolving market, where every single second matters in launching your product, releasing new features, and versions before everyone else.

Contents

Chapter 1

Introduction

1.1 Context

The modern cloud computing landscape relies heavily on microservices and containerization to deliver scalable, resilient applications. Docker and the Open Container Initiative (OCI) standards, frequently paired with mature backend languages like Golang, have become the de facto foundation for these deployments within Kubernetes environments [1], [2]. This traditional cloud-native stack prioritizes a frictionless developer experience and mature tooling. However, the growing demand for edge computing, high-density serverless functions, and instantaneous scaling has exposed the architectural limitations of conventional Linux containers. OCI containers rely on operating-system-level virtualization, which carries a baseline overhead in both memory consumption and initialization time [3].

As organizations seek to maximize hardware utilization and minimize latency, a shift toward more lightweight execution environments is underway. WebAssembly (Wasm), originally designed as a highly optimized, stack-based virtual machine binary format for web browsers [4], has emerged as a potential solution for server-side workloads. With the introduction of the WebAssembly System Interface (WASI), Wasm promises a portable, secure, and near-instantaneous execution model that operates without the heavy dependencies of a traditional Linux container [5].

1.2 Problem Statement

Standard OCI containers introduce measurable overhead regarding memory footprint and cold-start latency. While negligible for traditional monolithic applications, this overhead becomes a severe limiting factor for transient microservices that require rapid, sub-millisecond scaling. The emerging WebAssembly stack theoretically solves this by utilizing software-based fault isolation (SFI) to strip away the operating system layer entirely, offering drastically reduced image sizes and high-density execution [6].

However, transitioning from a mature ecosystem to a nascent one introduces substantial, often under-documented friction. The core problem this thesis addresses is the tension between computational efficiency and developer velocity. While the Wasm ecosystem promises high performance for server-side workloads, it currently lacks robust multithreading capabilities, enforces strict limitations on compiled network dependencies, and demands specialized expertise in systems languages like Rust.

There is a critical need to empirically quantify the holistic trade-offs of these two paradigms. It must be determined whether the theoretical efficiency gains of the Wasm stack outweigh the practical performance degradations and the operational complexities it introduces compared to the industry-standard Docker baseline.

1.3 Research Questions

To address the problem statement, this thesis investigates the following primary research questions:

- **RQ1:** How does the memory footprint and image size of the emerging Wasm stack compare to a traditional Docker baseline in a Kubernetes environment?
- **RQ2:** To what extent do WebAssembly runtimes improve cold and warm startup latencies compared to standard OCI container runtimes executing garbage-collected binaries?
- **RQ3:** What are the runtime performance trade-offs—specifically regarding request throughput and execution latency—introduced by WebAssembly's current constraints on multithreading compared to Golang's native concurrency model?
- **RQ4:** How do ecosystem limitations, specifically regarding the Rust learning curve, WASI compatibility, and CI/CD integration, impact the developer experience and maintainability when migrating from a traditional containerized pipeline?

1.4 Research Methodology

This research employs a mixed-methods approach, combining quantitative performance benchmarking with a qualitative assessment of developer experience.

Rather than isolating the runtime—which would necessitate comparing equivalent languages and thereby destroying the ability to measure real-world developer experience—this study compares the two dominant paradigms in their native contexts. First, a baseline microservice is developed using Golang and compiled into a standard Linux-based Docker image. Second, a functionally equivalent alternative is developed using Rust and compiled into a WASI-compliant WebAssembly module. Both artifacts are deployed into an isolated Kubernetes test cluster utilizing `containerd`, standard `runc`, and a Wasm shim.

Quantitative data is gathered by subjecting both deployments to standardized load testing. Monitoring tools are used to measure cold-start times, memory consumption, CPU utilization, throughput, and request latency. While this approach acknowledges that language-level differences (e.g., Go's garbage collection versus Rust's manual memory management) influence the final metrics, it is strictly designed to reflect the real-world performance delta an engineering team experiences when transitioning between these technological stacks. Second, a qualitative analysis is conducted on the development lifecycle, documenting compilation friction, library compatibility, and pipeline integration complexity.

1.5 Thesis Contributions

This thesis provides the following key contributions to the field of cloud-native architecture:

1. **Holistic Paradigm Benchmark:** A side-by-side performance comparison of the Golang/Docker stack versus the Rust/Wasm stack, highlighting specific metrics where the Wasm ecosystem succeeds (density, initialization) and where it currently fails (throughput, computational latency).
2. **Developer Experience Assessment:** A critical evaluation of the current state of WebAssembly for backend development, quantifying the engineering friction of compiling Rust code to WASI and navigating incompatible library dependencies compared to standard Go development.
3. **Adoption Framework:** Actionable recommendations for cloud architects, delineating specific use cases where the Rust/Wasm stack is currently viable and identifying the operational risks that make it unsuitable for rapid-iteration product teams accustomed to the Go/Docker ecosystem.

1.6 Thesis Structure

The remainder of this thesis is structured as follows:

- **Chapter 2: Background** provides the foundational technical context on cloud computing paradigms, containerization technologies, the Go concurrency model, and the evolution of WebAssembly and WASI.
- **Chapter 3: State of the Art** reviews the theoretical foundations of Wasm's security, the emerging Component Model, and current industry adoption.
- **Chapter 4: Research Methodology** defines the comparative analysis strategy, establishing the Golang/Docker baseline against the Rust/Wasm challenger, and outlines the precise metrics.
- **Chapter 5: Implementation** details the system architecture, service design, compilation processes, and Kubernetes cluster configuration for both environments.
- **Chapter 6: Evaluation** presents the quantitative benchmark results alongside the qualitative assessment of the developer experience.
- **Chapter 7: Discussion and Future Work** analyzes the trade-offs between technological maturity, computational efficiency, and developer velocity.
- **Chapter 8: Conclusion** summarizes the core findings and final verdicts of the research.

Chapter 2

Background

This chapter provides the foundational technical context required to evaluate the comparative analysis presented in this thesis. It traces the evolution of computing infrastructure from on-premises hardware to modern cloud paradigms, details the mechanics of containerization and orchestration, and culminates in a comprehensive architectural breakdown of WebAssembly as an emerging server-side execution environment.

2.1 Cloud Computing and Microservices

2.1.1 The Evolution of Cloud Paradigms

The genesis of modern software deployment traces back to on-premises data centers, where organizations were solely responsible for the procurement, maintenance, and scaling of physical hardware. This capital-heavy model suffered from severe resource underutilization; servers were provisioned for peak load, leaving them idle during standard operations. The modern cloud computing era shifted the industry to a pay-as-you-go, scalable operational model, fundamentally altering how infrastructure is managed [7].

As cloud computing matured, the industry developed a "Shared Responsibility Model," where the abstraction layer moved progressively higher up the stack. This spectrum dictates whether the cloud provider or the client is responsible for specific layers of the technological stack:

- **Infrastructure as a Service (IaaS):** The cloud provider manages the physical hardware, storage, and networking. The client rents virtualized infrastructure (e.g., a Virtual Machine) and retains full responsibility for managing the operating system, middleware, language runtimes, and the application code itself. This offers maximum control but requires significant operational overhead.
- **Platform as a Service (PaaS):** The provider abstracts both the underlying hardware and the operating system. The client is provided a managed environment and is solely responsible for deploying their application code and managing their data. Traditional Docker container deployments often operate in this space.
- **Function as a Service (FaaS) / Serverless:** The highest level of *infrastructure* abstraction for a developer. The cloud provider dynamically manages the allocation, provisioning, and scaling of the servers and runtimes. The client writes granular, event-driven functions and dictates the trigger configurations. The client is still strictly responsible for the software architecture and

code quality. Crucially, FaaS allows applications to "scale to zero," eliminating compute costs during periods of inactivity [8].

- **Software as a Service (SaaS):** The highest level of *software* abstraction. The cloud provider hosts and manages the entire application stack—from the hardware and OS up to the application code itself. The client manages nothing regarding the deployment or execution environment; they simply consume the finished software over the internet (e.g., Google Workspace, Salesforce).

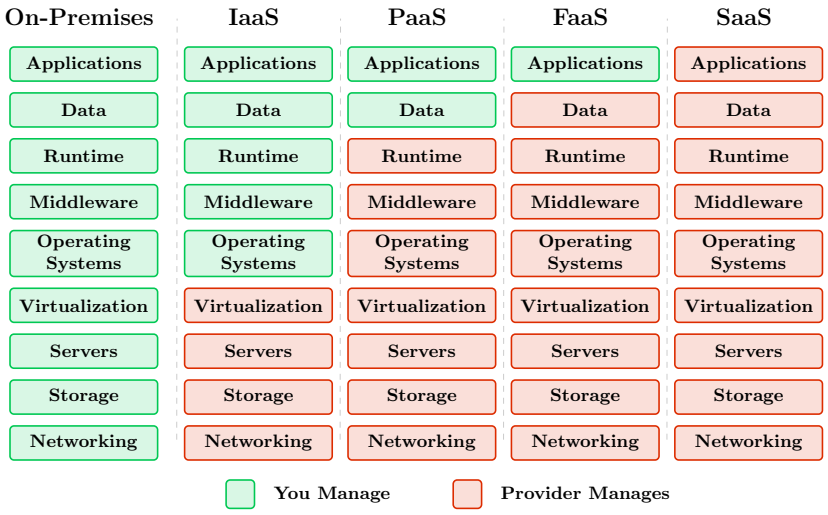

Figure 2.1 – The Shared Responsibility Model across varying Cloud Computing Paradigms.

For the purposes of this research, SaaS falls outside the scope of architectural implementation. The critical transition lies between PaaS and FaaS. The relentless industry drive toward rapid elasticity, microsecond billing, and extreme resource efficiency in FaaS and edge environments exposed the limitations of traditional, heavy deployment artifacts. Developers pushing code to serverless platforms require instantaneous execution. Standard OCI containers, laden with operating system userlands, introduce severe latency bottlenecks in these paradigms, necessitating a shift toward the lightweight isolation models that WebAssembly provides.

2.1.2 Microservice Architecture

Microservice architecture emerged as the logical software design pattern for cloud-native deployment. Unlike traditional monolithic applications—where all business logic, user interfaces, and database connections are tightly coupled into a single executable—microservices decompose an application into a collection of small, independently deployable, and loosely coupled services that communicate over a network via language-agnostic APIs [9].

This paradigm offers distinct advantages: development agility, localized fault isolation, and the ability to scale specific system bottlenecks independently. However, it introduces severe distributed system challenges. Managing network latency, data consistency, and the operational overhead of packaging and deploying hundreds of individual services necessitated the creation of entirely new virtualization and orchestration layers.

2.2 Containerization and Orchestration

2.2.1 Virtual Machines vs. Containers

To solve the deployment complexity of microservices, the industry initially relied on Virtual Machines. However, VMs require a hypervisor to emulate physical hardware, atop which a complete guest OS is installed. This results in gigabytes of memory overhead and startup times measured in minutes, making them inherently unsuitable for transient, rapidly scaling microservices.

Docker revolutionized the landscape by democratizing Linux containers. Instead of virtualizing the hardware, containers virtualize the operating system [1]. Docker relies on two core Linux kernel features:

1. **Namespaces:** Provide process isolation, ensuring the container has its own isolated filesystem, network interfaces, and process tree.

2. **Cgroups (Control Groups):** Limit, isolate, and monitor the resource usage (CPU, memory, disk I/O) of a given process.

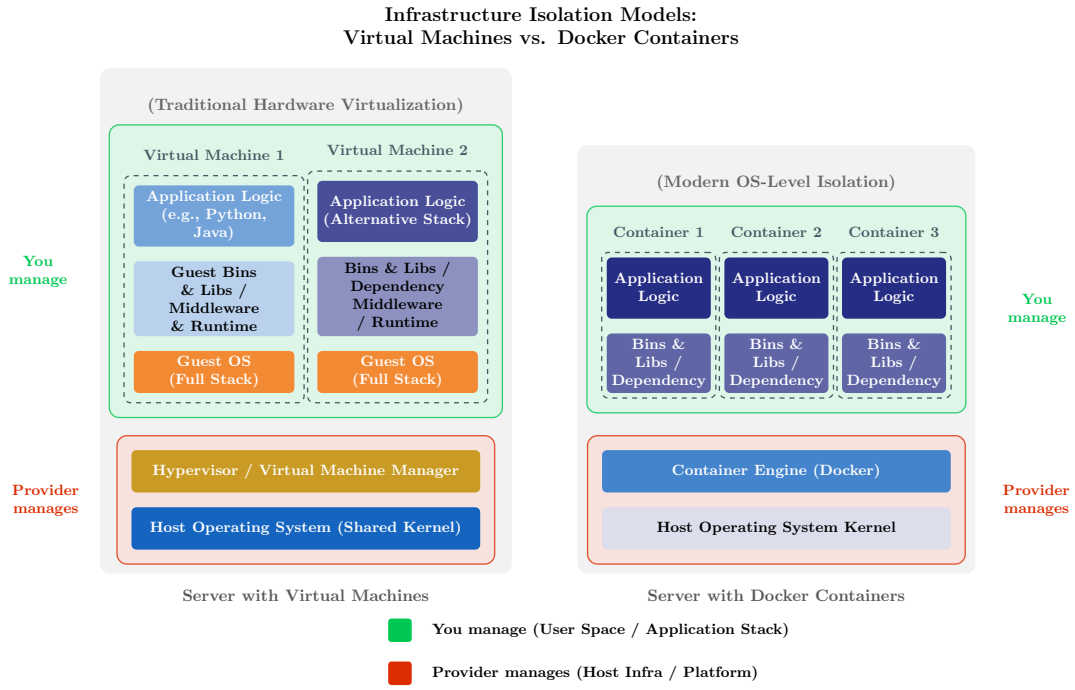

Figure 2.2 – Infrastructure Isolation Models: Virtual Machines vs. Docker Containers

Docker's definitive innovation was the standardization of the container image format, now governed by the Open Container Initiative (OCI). By packaging application code alongside its dependencies, libraries, and a base filesystem into a single portable unit, Docker ensured environment consistency from a developer's laptop to production servers.

Despite its dominance, the OCI standard has inherent drawbacks. Because containers share the host's Linux kernel, a kernel-level vulnerability can potentially compromise all containers on that host, posing security risks in multi-tenant environments. Furthermore, traditional container images are

bulky; they bundle a userland OS and language runtimes alongside the application. This bulk introduces latency during image transfer and initialization—known as the "cold start" problem—which is highly detrimental in FaaS and edge computing environments [6].

2.2.2 Kubernetes Orchestration

As microservices proliferated, manually managing individual containers across a fleet of servers became operationally unfeasible. Kubernetes emerged as the industry-standard container orchestration platform. It abstracts the underlying cluster of hardware, allowing engineers to declare the desired state of their applications rather than manually scripting deployments [2].

Kubernetes operates on a distributed architecture. The Control Plane dictates instructions to worker nodes. Each worker node runs an agent called a Kubelet, which manages Pods—the smallest deployable computing unit, typically containing one or more tightly coupled containers. Crucially, Kubernetes communicates with the underlying container runtime (like containerd or CRI-O) via the Container Runtime Interface (CRI). This decoupled architecture established strict standardizations, eventually allowing alternative, non-Linux execution environments, such as WebAssembly, to be orchestrated seamlessly alongside standard OCI-compliant containers.

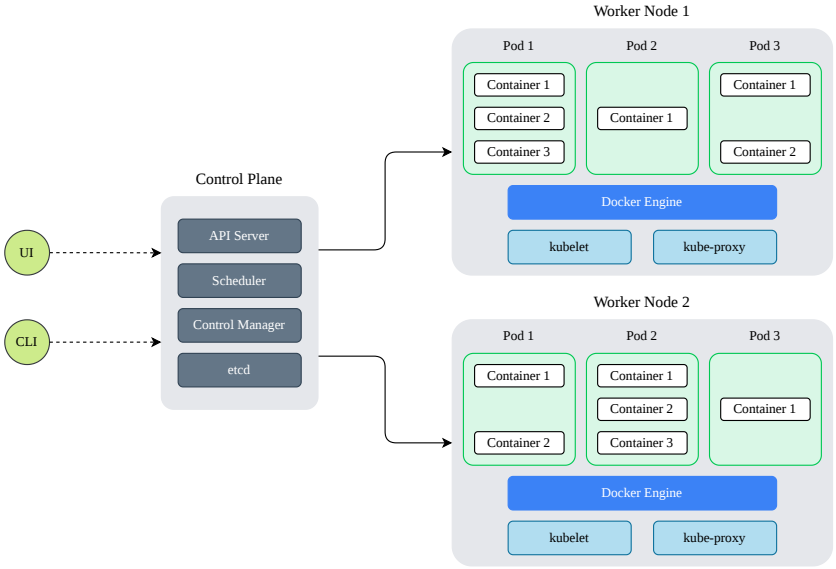

Figure 2.3 – Kubernetes Architecture

2.3 WebAssembly (Wasm)

2.3.1 Origin and Technical Architecture

WebAssembly (Wasm) is a binary instruction format designed for a stack-based virtual machine. It originated from the need to execute high-performance code within web browsers—a domain historically constrained by JavaScript's inconsistent performance and parsing overhead. WebAssembly was officially released in 2017 as a W3C standard, sitting alongside HTML, CSS, and JavaScript as an official language of the web [4].

Wasm functions as an agnostic hardware abstraction layer. Developers write code in systems languages like C, C++, or Rust, and compile it down to `.wasm` binaries.

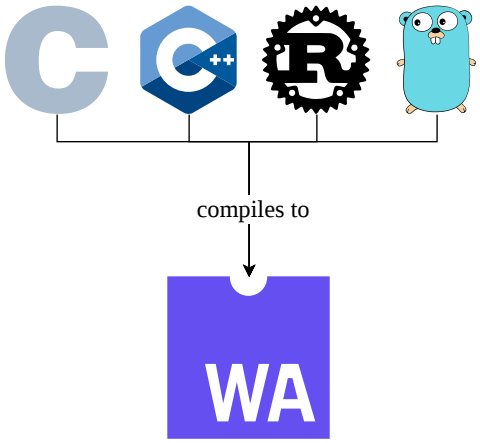

Figure 2.4 – Compiling different languages to Wasm

2.3.2 Module Structure and Stack-Based Execution

WebAssembly is fundamentally a stack machine. Unlike register-based physical CPU architectures (such as x86_64 or ARM64), Wasm instructions implicitly operate on an operand stack. Variables are pushed onto the stack, and operations pop these operands, compute the result, and push it back. This design yields highly compact binary modules, as instructions do not need to explicitly encode register numbers, accelerating both transmission over networks and decoding phases.

A Wasm application is distributed as a strictly standardized binary module consisting of discrete sections. These include the Type section (function signatures), Import section (host-provided dependencies), Code section (executable bytecode), and Data section (initial memory state). Before execution, a WebAssembly runtime performs a deterministic, single-pass validation to ensure type safety and structural integrity. Because the bytecode is designed to map cleanly to native machine instructions, runtimes employ Just-In-Time (JIT) or Ahead-Of-Time (AOT) compilation engines (such as Cranelift or LLVM) to translate Wasm into highly optimized, native machine code instantaneously.

2.3.3 Memory Management and Isolation

One of WebAssembly's defining technical characteristics is its approach to memory management and fault isolation. The architecture relies on the following core principles:

- **Compact Binary Format:** The `.wasm` format is highly optimized, allowing for rapid fetching, decoding, and execution.
- **Hardware Agnosticism:** Because it utilizes a virtual stack machine, the compiled bytecode makes no assumptions about the underlying physical CPU architecture, enabling true portability across varied cloud node pools.
- **Linear Memory Space:** Wasm operates in a contiguous, resizable array of bytes known as linear memory. Code executing within the Wasm module operates strictly with integer offsets within this

array; it cannot execute arbitrary pointers or access host memory outside this strictly enforced boundary.

- **Software-Based Fault Isolation (SFI):** Unlike Docker, which relies on the host OS kernel (cgroups/namespaces) to contain a process, Wasm achieves isolation mathematically at the runtime level. Any attempt to access memory outside the allocated linear boundary results in a deterministic trap, halting the module instantly without compromising the host.

2.3.4 Language Agnosticism and the JVM Comparison

The concept of "Write Once, Run Anywhere" (WORA) was popularized by Java. Java achieves portability by compiling source code to Java bytecode, which is executed by the Java Virtual Machine (JVM). However, WebAssembly fundamentally differs from the JVM in design and scope. The JVM was built specifically to execute Java and inherently carries a heavyweight runtime featuring built-in garbage collection. Conversely, Wasm was designed as a low-level, truly language-agnostic compilation target without an opinionated runtime, making it exceptionally lightweight.

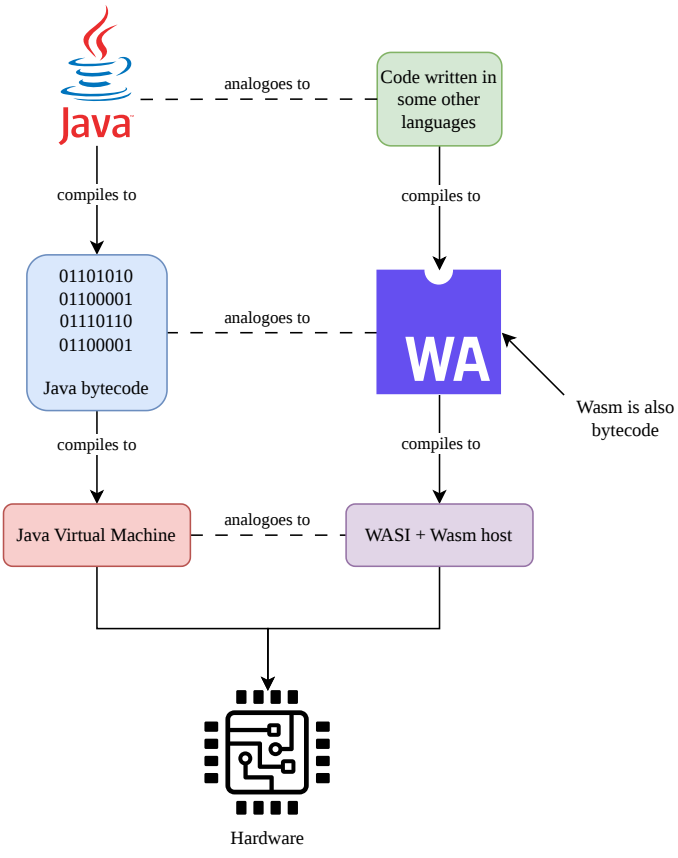

Figure 2.5 – An analogy comparing the cross-platform execution models of Java and WebAssembly. Adapted from Chiarlone [10].

For languages requiring automatic memory management (like Go, Python, Kotlin, or PHP), Wasm historically required the entire language runtime and garbage collector to be compiled and bundled

inside the `.wasm` binary. This approach bloated the deployment artifact and created redundant layers of memory management. For instance, deploying a compiled-to-Wasm language with its own garbage collector into a host environment that already possesses one (such as a browser engine like V8) is highly wasteful [11]. Furthermore, having no interaction between Wasm's linear memory and the built-on-top garbage collector of the ported language often caused problems like memory leaks and inefficient collection attempts [12].

To resolve this, the WebAssembly Garbage Collection (WasmGC) standard has been introduced to delegate memory management directly to the host environment. WasmGC extends the WebAssembly memory model by adding struct and array heap types, thereby providing native support for non-linear memory allocation [13]. Because each WasmGC object has a fixed type and structure, host VMs can generate highly efficient access code without the risk of dynamic deoptimizations. This allows higher-level managed languages to bypass bundling their own garbage collectors, instead interfacing efficiently with the host VM's built-in garbage collector.

The architectural impact on artifact density is significant. By eliminating the need to bundle memory management code, WasmGC binaries for managed languages can achieve dramatically smaller footprints. In early benchmarks, WasmGC binaries for languages like Java were proven to be considerably smaller than equivalent modules compiled from C or Rust, because C and Rust must still bundle manual memory management routines like `malloc` and `free` inside the binary [11]. This advancement firmly positions Wasm as a viable, high-density target for the entire spectrum of modern programming languages.

2.3.5 The Server-Side Evolution: WASI

While Wasm successfully delivered near-native performance for computationally heavy browser applications, its portability, mathematical security model, and minimal footprint rapidly garnered interest for backend deployment. Solomon Hykes, the founder of Docker, once famously stated:

> If WASM+WASI existed in 2008, we wouldn't have needed to create Docker. That's how important it is. WebAssembly on the server is the future of computing. [14]

Because Wasm is entirely isolated by default, a bare Wasm module running on a server cannot open a network socket, read a file, or query the system clock. In 2019, the WebAssembly System Interface (WASI) was introduced to solve this. WASI defines a standardized API specification that provides Wasm modules with secure, granular, capability-based access to the host operating system [15]. By acting as a secure POSIX-like layer, WASI transforms Wasm from a sandboxed browser calculator into a fully functional server-side technology.

2.3.6 Capability-Based Security Model

WASI implements a capability-based security model that diverges significantly from traditional Linux permissions. A Linux container limits an already-running OS environment. WASI, however, enforces a strict "deny-by-default" posture.

If a Wasm module needs to access a configuration file, it cannot invoke a standard ‘open()‘ system call for an arbitrary path (e.g., ‘/etc/config‘). Instead, the host runtime must explicitly grant the module a "pre-opened" file descriptor at startup. The module is completely blind to the existence of the host filesystem outside the specific directory tree explicitly mapped and passed to it by the runtime.

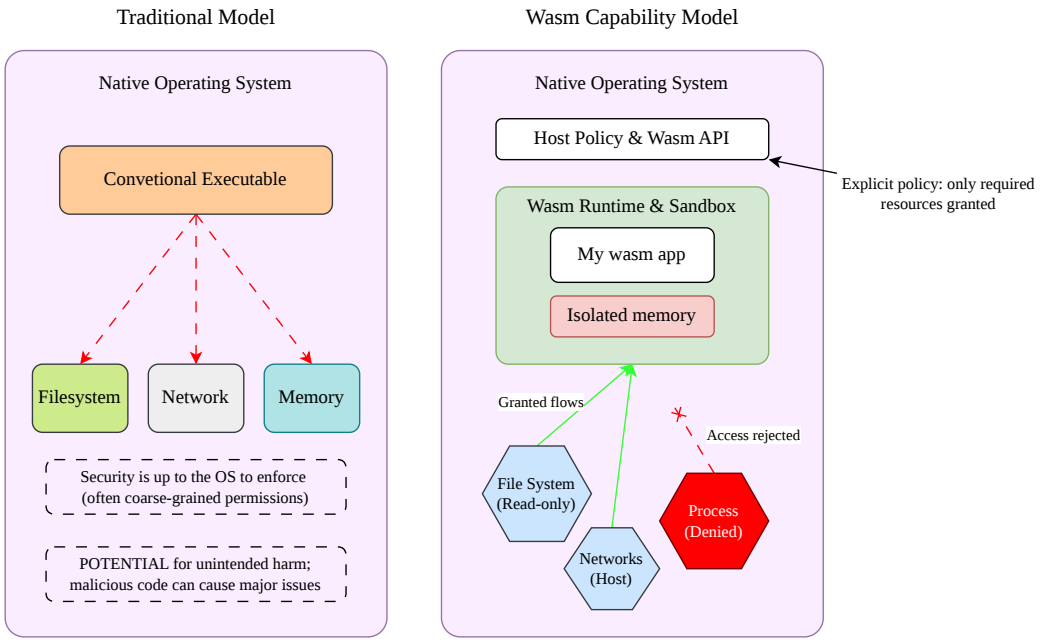

Figure 2.6 – Comparative Security Models: Traditional vs Wasm Capabilities

2.3.7 Architectural Evolution: From POSIX to the Component Model

The initial iteration of WASI (often referred to as WASI Preview 1) was heavily modeled after standard POSIX interfaces, focusing on file descriptors, environment variables, and standard I/O streams. While effective for simple command-line tools and basic microservices, this POSIX-centric approach introduced friction when handling complex, multi-language cloud-native interactions.

To address this, the WebAssembly ecosystem is currently transitioning to the WebAssembly Component Model (WASI Preview 2 and Preview 3). The Component Model shifts WASI from a low-level POSIX API to a high-level, language-agnostic interface system utilizing WebAssembly Interface Types (WIT). This advancement allows discrete Wasm components—potentially written in entirely different source languages—to natively interoperate. They can securely pass complex data types (such as strings, records, and variants) across execution boundaries without sharing linear memory, laying the foundation for highly modular, secure, and dynamically composable microservice architectures.

2.3.8 Wasm vs. Traditional Containers

Applying Wasm and WASI to the server yields a deployment artifact that sits functionally between a traditional VM and an OCI container, offering unique operational characteristics:

1. **Cold Start and Footprint:** Traditional OCI containers bundle an OS userland, frequently resulting in image sizes exceeding 100MB and startup times measured in seconds. A Wasm module contains only the compiled application logic. Wasm binaries are typically just a few megabytes and can be instantiated by a host runtime in microseconds. Long et al. demonstrated that Wasm runtimes can perform at least 10x faster than Docker from a cold start for short-running workloads [16]. Hall and Ramachandran further verified that Wasm's improved cold start times lead to up to 90% faster response times for basic serverless tasks [17].

2. **Security and Density:** Wasm utilizes Software-Based Fault Isolation (SFI). If a Wasm module is compromised, the attacker only controls the isolated linear memory sandbox. Because Wasm threads do not require full OS process overhead, the minimal memory footprint allows Kubernetes nodes to achieve exponentially higher workload density.

3. **Performance Trade-offs:** Despite its advantages in density and initialization, Wasm remains structurally immature compared to Docker regarding concurrency. Wasm currently lacks robust, standardized multithreading capabilities (the Wasm Threads proposal relies on shared memory buffers that complicate isolation). Sondell (2024) demonstrated that in high-concurrency HTTP server scenarios, a Docker container can process requests significantly faster because it can scale natively across multiple CPU cores, whereas a single Wasm instance frequently remains bottlenecked by single-threaded execution [18]. Furthermore, Spies and Mock found that single-threaded Wasm apps perform approximately 14% slower than native x86_64 binaries due to sandbox bounds-checking overhead [19].

By utilizing WASI shims (such as `runwasi`), Kubernetes can now orchestrate these high-density Wasm workloads directly alongside traditional Docker containers, establishing the precise technological foundation for the comparative analysis executed in this research.

Chapter 3

State of the Art and Industry Landscape

This chapter reviews the theoretical foundations of server-side WebAssembly and surveys the current industrial adoption of Wasm as a microservice runtime. It contrasts academic findings on isolation and performance with real-world platforms provided by major tech vendors.

3.1 Theoretical Foundations of Server-Side WebAssembly

3.1.1 The Wasm Security Model vs. Container Isolation

An analysis of the "capability-based" security model of WebAssembly (WASI) compared to the Linux namespace and cgroup isolation used by Docker. Discussion includes memory safety, control-flow integrity, and the reduction of the attack surface.

3.1.2 Performance Characteristics in Academia

A review of academic benchmarks comparing Wasm execution speeds (JIT vs. AOT compilation) against native code and interpreted languages (like Golang). Focuses on "Cold Start" latency and memory density.

3.2 The WebAssembly Component Model

Introduction to the Wasm Component Model and WIT (Wasm Interface Type), which solves the "Nano-process" communication problem, allowing microservices to compose without network overhead—a theoretical advantage over traditional HTTP-based microservices.

3.3 Industry Adoption and Ecosystem

3.3.1 Edge Computing Providers

- **Cloudflare Workers:** Utilization of V8 Isolates for near-instant startup.
- **Akamai and Linode:** Integration of Wasm for edge compute, highlighting their recent acquisition of Fermyon technologies.

- **Fastly Compute:** Use of the Lucet (now Wasmtime) runtime for sub-millisecond instantiation.

3.3.2 Application Frameworks

- **Fermyon Spin:** A developer-centric framework for building event-driven microservices.
- **WasmCloud:** An actor-model-based platform focusing on distributed systems and "location transparency".
- **WasmEdge:** A CNCF sandbox project optimized for AI/ML workloads and extension of cloud-native ecosystems.

3.4 WebAssembly in Kubernetes

3.4.1 Evolution of Integrations

From the deprecated *Krustlet* project to modern *containerd* shims (runwasi).

3.4.2 Modern Operators

- **SpinKube:** The de-facto standard for running Spin apps in K8s.
- **WasmCloud Operator:** Bridging Kubernetes orchestration with the WasmCloud lattice.
- **KWasm:** An operator that automates the installation of Wasm shims on K8s nodes.

Chapter 4

Research Methodology

4.1 Research Strategy: Comparative Analysis

This thesis employs a quantitative comparative analysis (A/B testing) to evaluate the suitability of Wasm/Rust against Docker/Golang.

4.2 Selection of Technologies

4.2.1 The Baseline: Docker + Golang

Justification for choosing Golang (FastAPI/Flask) as the baseline: it represents the industry standard for rapid development but suffers from high memory usage and slow startup (the "Garbage Collection" penalty).

4.2.2 The Challenger: WebAssembly + Rust

Justification for Rust: it provides the smallest Wasm binary sizes and lacks a garbage collector, maximizing the potential benefits of the Wasm runtime.

4.2.3 Orchestration: Kubernetes with RuntimeClass

Description of the hybrid cluster setup where both artifacts will run side-by-side.

4.3 Definition of Metrics

- **Startup Latency (Cold Start):** Time from ‘kubectl scale‘ to the first HTTP 200 OK response.
- **Memory Footprint:** RSS (Resident Set Size) memory usage at idle and under load.
- **Artifact Size:** Comparison of the OCI image size (Golang slim layers) vs. the Wasm module size.
- **Throughput Latency:** Requests Per Second (RPS) and P99 latency under sustained load (using tools like K6 or Hey).
- **Maintainability (Qualitative):** Assessment of the "Inner Development Loop," debugging difficulty, and CI/CD complexity.

Chapter 5

Implementation of the Microservice Prototype

5.1 System Architecture

Description of the "Reference Microservice" logic (e.g., a stateless service that performs image processing or cryptographic calculations to stress CPU and Memory).

5.2 Baseline Implementation (Golang/Docker)

5.2.1 Service Design

Implementation details using Golang (FastAPI).

5.2.2 Container Optimization

Efforts to minimize the Docker image (e.g., using Alpine Linux or Distroless images) to ensure a fair comparison.

5.3 Experimental Implementation (Rust/Wasm)

5.3.1 Service Design

Re-implementation of the logic in Rust using the ‘spin-sdk‘ or ‘wasmcloud-actor‘ SDK.

5.3.2 Compilation and Packaging

Compiling to ‘wasm32-wasi‘ and packaging as an OCI artifact.

5.4 Kubernetes Cluster Configuration

5.4.1 Runtime Class Setup

Configuring ‘containerd‘ to use ‘runwasi‘ or ‘spin-shim‘ for specific workloads.

5.4.2 Ingress and Networking

Configuring Traefik or Nginx to route traffic to both the Wasm and Docker services.

Chapter 6

Evaluation

6.1 Benchmark Environment

Hardware specifications of the Kubernetes cluster (local Minikube vs. Managed Cloud K8s) and the load testing tools used.

6.2 Results: Startup Performance

Presentation of data showing the "Cold Start" gap between the Golang VM initialization vs. Wasm JIT instantiation.

6.3 Results: Resource Efficiency

Analysis of memory density. (Expected Result: Wasm modules using <5MB vs Golang containers using >50MB).

6.4 Results: Runtime Performance

Comparison of raw CPU throughput. Discussion on whether the lack of Garbage Collection in Rust/Wasm offsets the overhead of the Wasm runtime sandbox compared to Native Golang.

6.5 Qualitative Analysis: Developer Experience

Documentation of the challenges faced during the Rust migration and K8s integration (e.g., debugging Wasm modules, ecosystem maturity).

Chapter 7

Discussion and Future Work

7.1 The Trade-off: Maturity vs. Efficiency

Discussion on the "Ecosystem Gap." While Wasm is more efficient, Golang/Docker has vastly superior library support and tooling availability.

7.2 Security Implications

How the "deny-by-default" security posture of Wasm changes the cluster security model compared to root-access risks in Docker.

7.3 Migration Recommendations

Guidelines for companies (like Akamai or Cloudflare customers) on when to switch to Wasm (e.g., high-scale serverless functions) versus staying with Docker (stateful monoliths).

Chapter 8

Conclusion

Summary of findings confirming whether Wasm/Rust serves as a viable alternative for the specific use case of high-density microservices.

Appendix A

Reproducibility

A.1 Abstract

A.2 Artifact check-list (meta-information)

A.3 Description

A.3.1 How to access

A.3.2 Hardware dependencies

A.3.3 Software dependencies

A.4 Installation

A.5 Experiment workflow

A.6 Evaluation and expected results

A.7 Notes

List of Abbreviations

List of Figures

List of Tables

Bibliography

[1] D. Merkel, "Docker: lightweight linux containers for consistent development and deployment," *Linux journal*, vol. 2014, no. 239, p. 2, 2014.

[2] B. Burns, B. Grant, D. Oppenheimer, E. Brewer, and J. Wilkes, "Borg, Omega, and Kubernetes," *Communications of the ACM*, vol. 59, no. 5, pp. 50–57, 2016.

[3] S. Shillaker and P. Pietzuch, "Faasm: Lightweight isolation for efficient stateful serverless computing," in *2020 USENIX Annual Technical Conference (USENIX ATC 20)*, 2020, pp. 419–433.

[4] A. Haas, A. Rossberg, D. L. Schuff, B. L. Titzer, M. Holman, D. Gohman, L. Wagner, A. Zakai, and J. Bastien, "Bringing the web up to speed with WebAssembly," in *Proceedings of the 38th ACM SIGPLAN Conference on Programming Language Design and Implementation*, 2017, pp. 185–200.

[5] M. Lin et al., "WasmEdge: WebAssembly for the Edge," *IEEE Software*, vol. 39, no. 2, pp. 91–96, 2022.

[6] P. K. Gadepalli, G. Peach, L. Cherkasova, R. Aitken, and G. Parmer, "Challenges and opportunities for efficient serverless computing at the edge," *Proceeding of the 38th Symposium on Reliable Distributed Systems (SRDS)*, 2019.

[7] P. Mell, T. Grance, et al., "The NIST definition of cloud computing," 2011.

[8] E. Jonas, J. Schleier-Smith, V. Sreekanti, C.-C. Tsai, A. Khandelwal, Q. Pu, V. Shankar, J. Carreira, K. Krauth, N. Yadwadkar, et al., "Cloud programming simplified: A berkeley view on serverless computing," *arXiv preprint arXiv:1902.03383*, 2019.

[9] S. Newman, Building microservices: designing fine-grained systems. O'Reilly Media, Inc., 2015.

[10] D. Chiarlone, Server-Side WebAssembly. Shelter Island, NY: Manning Publications, 2024.

[11] T. Steiner. "WebAssembly Garbage Collection (WasmGC) now enabled by default in Chrome," Google Chrome Developers. (2023), [Online]. Available: `https://developer.chrome.com/blog/wasmgc` (visited on 03/04/2026).

[12] M. Liedtke, A. Klein, A. Zakai, et al. "A new way to bring garbage collected programming languages efficiently to WebAssembly," V8 JavaScript Engine. (2023), [Online]. Available: `https://v8.dev/blog/wasm-gc-porting` (visited on 03/04/2026).

[13] WebAssembly Community Group, WebAssembly Garbage Collection Proposal Overview, 2023.

[14] S. Hykes, If WASM+WASI existed in 2008, we wouldn't have needed to created Docker. Twitter, 2019.

[15] L. Clark. "Standardizing WASI: A system interface to run WebAssembly outside the web," Mozilla Hacks. (2019), [Online]. Available: `https://hacks.mozilla.org/2019/03/standardizing-wasi-a-webassembly-system-interface/`.

[16] X. Long et al., “A Performance Comparison of WebAssembly and Container Technologies,” in *Proceedings of Cloud Computing and Containerization Conferences*, 2022.

[17] A. Hall and U. Ramachandran, “Serverless Cold Start Mitigation with WebAssembly,” *IEEE Transactions on Cloud Computing*, 2021.

[18] E. Sondell, “Performance Evaluation of WebAssembly Containers vs Docker Containers for Serverless Applications,” M.S. thesis, KTH Royal Institute of Technology, 2024.

[19] A. Spies and M. Mock, “An evaluation of WebAssembly in non-web environments,” in *Proceedings of the 14th ACM International Conference on Systems and Storage*, 2021, pp. 1–11.